solve conflicts. Then, we transform each key into an index between `0` and `M - 1`, using a *hash function*.

It's worth noting that introducing such a transformation relaxes the constraint about having only non-negative integers keys: we can "serialize" any object into a string and transform any string into an integer modulo `M` as part of hashing. Next, we'll assume keys are integers for the sake of brevity.

The exact hash function we need to use depends on the type of the keys, and it's correlated to the size of the array. The most notable examples include:

- The *division method*: given an integer key `k`,

$$h(k) = k\%M$$

  (where % represents the modulo operator). For this method, `M` should be a prime number not too close to a power of 2.

  For instance, if `M=13`, we would have `h(0)=0, h(1)=1, h(2)=2,… h(13)=0, h(14)=1` and so on.

- The multiplication method:

$$h(k) = \lfloor M * (k * A\%1) \rfloor$$

  where `0 < A < 1` is a Real constant, and `(k*A % 1)` is the fractional part of `k*A`. In this case, `M` is usually chosen as a power of 2, but `A` is to be chosen carefully, depending on `M`.

  For instance, say M=16 and A=0.25, then

  $k = 0 \Rightarrow h(k) = 0$
  $k = 1 \Rightarrow k * A = 0.25, k*A\%1 = 0.25, h(k) = 4$
  $k = 2 \Rightarrow k * A = 0.5, k*A\%1 = 0.5, h(k) = 8$
  $k = 3 \Rightarrow k * A = 0.75, k*A\%1 = 0.75, h(k) = 12$
  $k = 4 \Rightarrow k * A = 1, k*A\%1 = 0, h(k) = 0$
  $k = 5 \Rightarrow k * A = 1.25, k*A\%1 = 0.25, h(k) = 4$

  … and so on

  As you can see, 0.25 was not a great choice for A, because this way h(k) will only assume 5 different values. To this extent, it was though